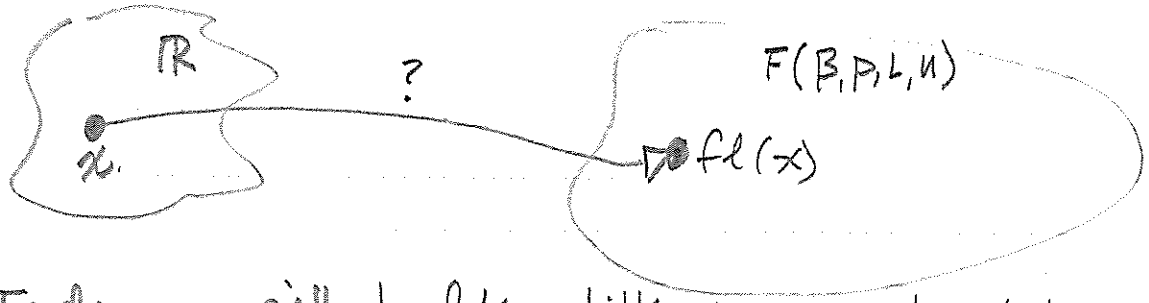
- today's office hour → Wed instead 3-4 BA 4230

Lecture 4
Mon 13 Jan 2025

Floating Point Number Systems.

Last time we noted that while we <u>want to</u> compute with $x \in \mathbb{R}$, what we <u>get to</u> compute with is $fl(x) \in F(\beta, p, L, U)$.



Today, we'll talk a little more about the properties of $F(\beta, p, L, U)$ and then will move to talk about the error made when approximating x by $fl(x)$.

Before the 1980's, many different choices for β, p, L, U (base, prec, exp range) were used by computer manufacturers. In the 1980's, the IEEE floating point standards in use today were approved, resulting in (ability to write) more portable s/w.

The person who spearheaded the creation of the IEEE floating point standard in the 1980's was William Kahan.

He was born in Toronto, studied at the U of T, was one of the first CS profs in the department, and moved to UC Berkeley in the 1960's.

He was awarded the ACM Turing Award in 1989 for his contributions to floating-point arithmetic.

His nickname is Velvel which means little wolf.

We have

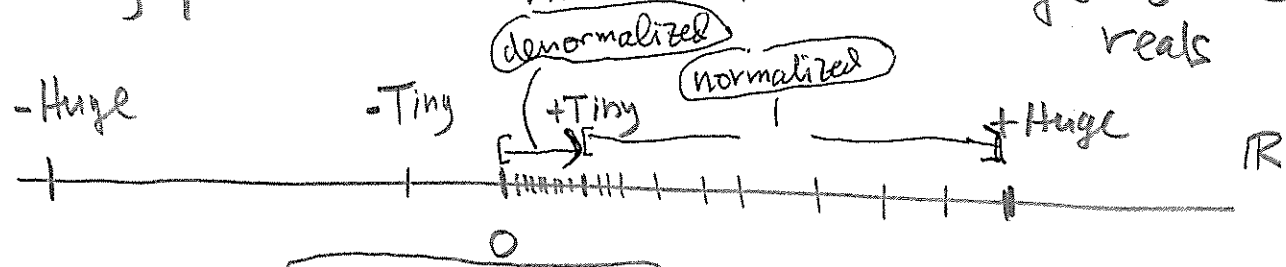
$$F(\beta, p, L, U) = \{ \pm d_0 . d_1 d_2 \ldots d_{p-1} \times \beta^E \}$$
$$\cup \{ \underset{E=L-1}{\text{denormalized}}, \underset{E=U+1}{\text{special}} \}$$

- a <u>finite</u> set with cardinality

$$\underset{\substack{\uparrow \\ \text{sign} \\ \text{choices}}}{2} \times \underset{\substack{\uparrow \\ d_0 \\ \text{choices}}}{(\beta - 1)} \times (\beta)^{p-1} \times \underset{\substack{\# \text{ exponents}}}{(U - L + 1)}$$

+ # denormalized and special cases

They provide a <u>non uniform</u> coverage of the reals



IEEE Standard

As mentioned

	single prec	double prec
β	2	2
p	24	53
L	-126	-1022
U	+127	+1023

Huge $\quad +1.111\ldots1 \times 2^{127}$ $\quad\doteq 3.4 \times 10^{38}$ $\qquad +1.111\ldots1 \times 2^{1023}$ $\quad\doteq 1.8 \times 10^{+308}$

Tiny $\quad +1.00\ldots0 \times 2^{-126}$ $\quad\doteq 1.2 \times 10^{-38}$ $\qquad +1.00\ldots0 \times 2^{-1022}$ $\quad\doteq 2.2 \times 10^{+308}$

power should be: -308

How should we handle $x \in \mathbb{R}$ that are > HUGE?

e.g. $F(10, 3, -10, 10)$ (classroom f.p. system)

$\text{HUGE} = +9.99 \times 10^{10}$

$x > \text{HUGE}$ means x <u>overflows</u> F.

We take $fl(x) = +\text{Inf} = 1.00\cdots0 \times \beta^{U+1}$

the floating point representation of x

Inf − Inf produces NotANumber NaN

use $1.xx\cdots x \; \beta^{U+1}$

some $d_i \neq 0$.

For $0 < x < \text{Tiny}$

take

$$fl(x) = \begin{cases} 0.0\cdots d_i d_{i+1}\cdots \times \beta^{L-1} & \text{gradual underflow until all } d\text{'s are } 0 \\ 0 & \text{eventually} \end{cases}$$

in either case

x underflows F.

next time: measuring the accuracy of representations

+ the accuracy of floating-point arithmetic

In class, we use simpler f.p. systems. Similar results will follow in IEEE f.p.

4

Suppose we use $F(\beta, p, L, u) = F(10, 3, -10, 10)$.

$$x = 4.00 \times 10^5 \quad \in F$$
$$y = 3.00 \times 10^5 \quad \in F$$

Compute $z = x*x + y*y \qquad (x^2+y^2)$

$$= 1.60 \times 10^{11} + 9.00 \times 10^{10}$$

$\hookrightarrow$ HUGE in $F(10,3,-10,10)$

We get $fl(x^2+y^2) = +\text{Inf}$, which is okay as the exact result overflows F.

Suppose instead we wanted to compute

$$h = \sqrt{x^2+y^2}$$

The correct result is $h = 5.00 \times 10^5$ which is in $F(10,3,-10,10)$ and so we ought to compute it.

If we used the algorithm: compute x^2 $\rightarrow$ Inf
— " — y^2
add $+$
compute $\sqrt{}$ using sqrt()

we get Inf!

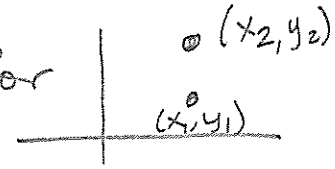
We need to modify the algorithm in order to compute an accurate result.

Compute $h = \max(|x|,|y|)\sqrt{1 + \left(\min(|x|,|y|)/\max(|x|,|y|)\right)^2}$

python math + other languages $\sim$ use math.hypot(x,y) as it uses this scaled algorithm

Sometimes computing Inf is the correct result.

e.g. slope $= (y_2 - y_1)/(x_2 - x_1)$ for



with $x_2 = x_1$

Inf slope just means parallel to the y axis.

(before 1980's, computing Inf could halt your program!)

For $x \in \mathbb{R}$, with $0 < x < TINY$, $fl(x) = 0$.

we underflow to 0.

Sometimes underflow can be okay.

Compute $y = 1 + x + (x^2/2)$ in $F(10, 3, -10, 10)$

with $x = 1.00 \times 10^{-5}$

get $y = 1.00 \times 10^{0} + 1.00 \times 10^{-5} + 5.00 \times 10^{-11}$

↳ underflows.

setting underflow to 0 does not introduce extra error result $y = 1.00 \times 10^{0}$ is accurate.

$\boxed{\text{but}}$ sometimes underflow to 0 can increase error

Consider $x = 5.00 \times 10^{-6}$, $y = 1.20 \times 10^{-5}$

$h = \text{sqrt}(x*x + y*y)$

$= \text{sqrt}(2.50 \times 10^{-11} + 1.44 \times 10^{-10})$

$< TINY$ - replace by 0

get $h = 1.20 \times 10^{-5}$

true $h = 1.30 \times 10^{-5}$ error ~ 8%.

So use hypot() function!